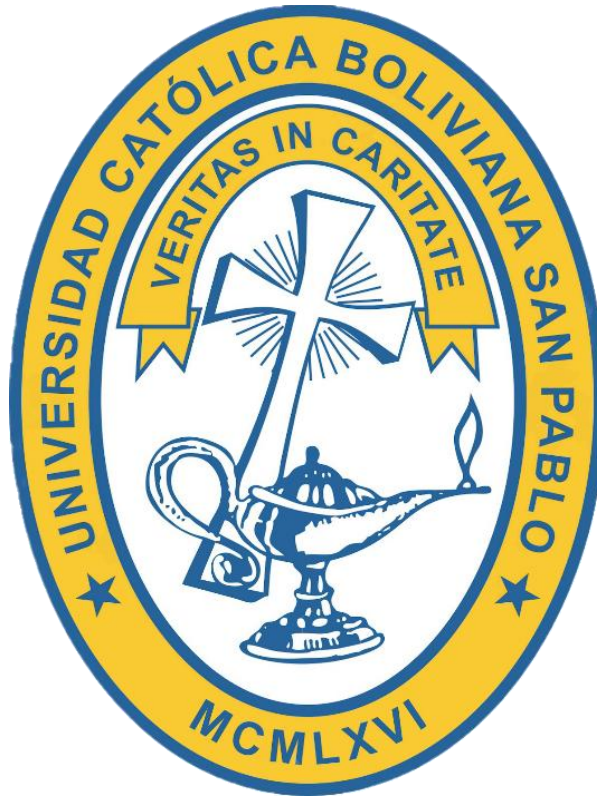


Universidad Católica Boliviana "San Pablo"



Sistema de Gestión de Proyecto Ser Familia

Documento 3 – Modelo C4

Materia: Ingeniería de Software

Estudiantes:

Escaray Rivero Paulo Roberto (director de Proyecto / Analista de Sistemas)

Murillo Ulloa Luis Marcel (especialista QA / Desarrollador Frontend)

Rueda Muñoz Daniel Boris (arquitecto de Software / Desarrollador Backend)

Semestre: 2 - 2026

Docente: M.Sc. Ing. Miguel Angel Pacheco Arteaga

La Paz – Bolivia

Contenido

Contenido.....	2
4. Introducción a la arquitectura.....	3
4.1 Propósito y alcance.....	3
4.2 Pila tecnológica.....	3
4.3 Diagramas C1 y C2	3
5. Diagrama de Componentes (C3) — API REST Core.....	5
5.1 Descripción de los componentes.....	6
5.2 Interacciones clave en tiempo de ejecución.....	7
5.3 Estructura de código propuesta	7
6. Registros de Decisiones Arquitectónicas (ADR)	8
ADR-001: Framework backend.....	8
ADR-002: Estrategia de persistencia	9
ADR-003: Almacenamiento de evidencias	9
ADR-004: Autenticación y autorización.....	10
ADR-005: Estilo arquitectónico de la API	11
ADR-006: Procesamiento asíncrono y notificaciones	12
ADR-007: Máquinas de estado para avances, tramos y proyectos	12

4. Introducción a la arquitectura

4.1 Propósito y alcance

En esta parte se documenta el tercer nivel del modelo C4 (Componentes) del contenedor **API REST**, junto con las decisiones arquitectónicas que lo sustentan. Los niveles C1 y C2 fueron elaborados previamente por el equipo y se toman como punto de partida; el C3 se centra en la API porque ahí vive la lógica de negocio, la seguridad y la integración con los sistemas externos. La arquitectura está pensada para poder desplegarse en un servidor con Docker Compose desde el primer día de desarrollo.

4.2 Pila tecnológica

Capa	Tecnología	Motivo
Frontend (SPA)	Vue 3 (Composition API), Vite, TypeScript, Vue Router, Pinia, PrimeVue, Axios	Componentes reutilizables, tipado compartido con la API, interfaz responsive (RNF-06).
Backend (API)	NestJS 11 sobre Node.js 22 LTS, TypeScript estricto	Modularidad e inyección de dependencias (ADR-001, ADR-005).
Persistencia	PostgreSQL 16, TypeORM con migraciones	Datos jerárquicos y transaccionales (ADR-002).
Evidencias	MinIO (S3-compatible), @aws-sdk/client-s3	Objetos fuera de la BD, URLs prefiradas (ADR-003).
Seguridad	Passport-JWT, bcrypt, @nestjs/throttler, Helmet	Autenticación y RBAC (ADR-004).
Asincronía	@nestjs/event-emitter, pg-boss, @nestjs/schedule	Notificaciones y reportes con reintentos sin contenedor extra (ADR-006).
Documentos	pdfmake (PDF), ExcelJS (XLSX), Nodemailer + Handlebars	Reportes y correos (RF-20, RF-21, RF-35).
Calidad y DevOps	Jest, Supertest, Playwright, k6, ESLint, GitHub Actions, Docker Compose, Nginx (TLS)	RNF-04, RNF-08, RNF-09.
Observabilidad	@nestjs/terminus (/health), logs estructurados JSON (pino)	RNF-07.

4.3 Diagramas C1 y C2

A continuación, se presentan los diagramas de Contexto (C1) y Contenedores (C2) del sistema. El C1 muestra los tres roles que interactúan con el SGP — Técnico de Campo, Director/Administrador y Financiador — junto con el Servidor de Correo como sistema externo. El C2 descompone el sistema en sus unidades desplegables: la SPA en Vue 3 con TypeScript, la API REST en NestJS, la base de datos PostgreSQL 16, el repositorio de evidencias en MinIO (S3-compatible) y el Servidor de Correo para las notificaciones.

C1 · Diagrama de Contexto — Sistema de Gestión de Proyectos Serfamilia

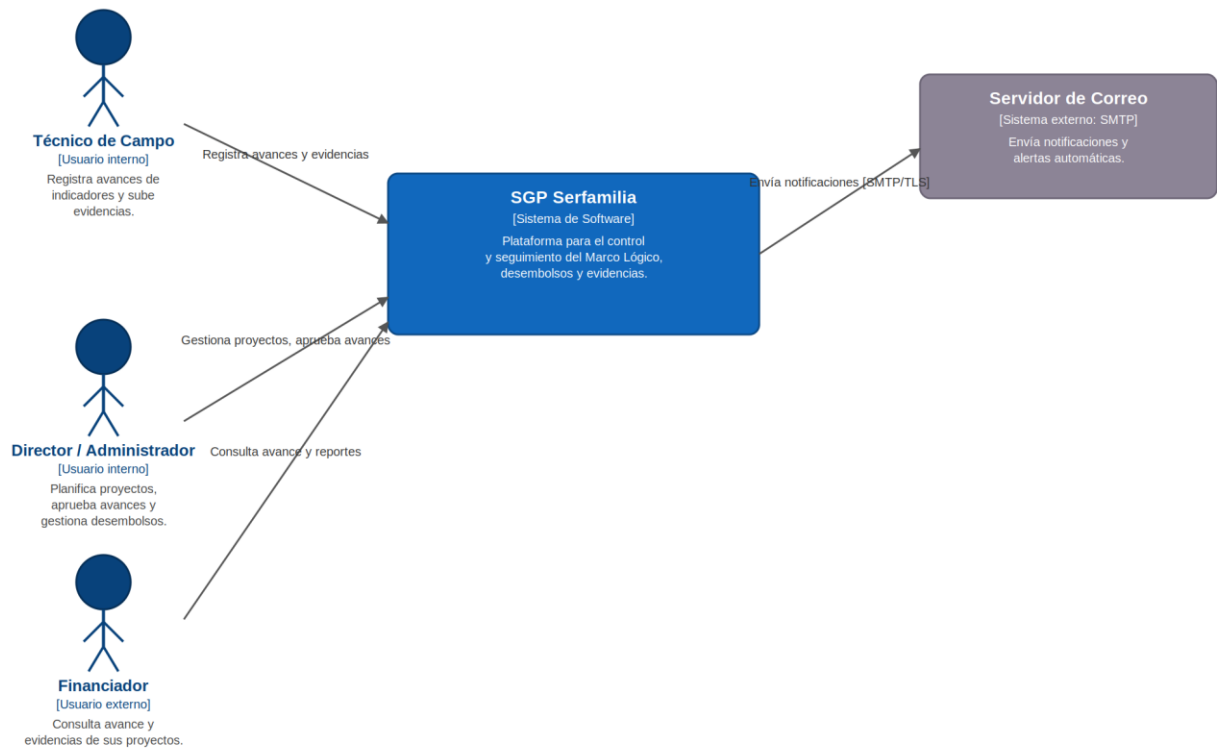


Figura 1. Diagrama de Contexto (C1). Incluye al Financiador y al Servidor de Correo externo.

C2 · Diagrama de Contenedores — Sistema de Gestión de Proyectos Serfamilia

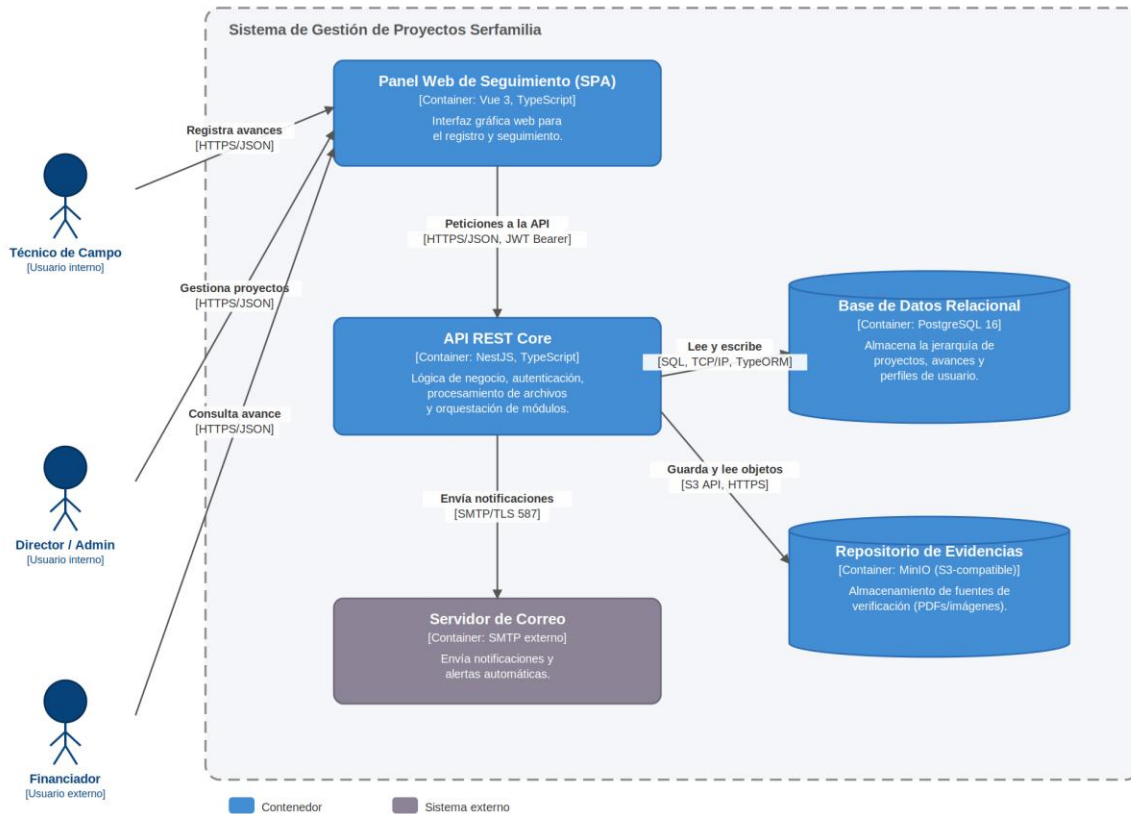


Figura 2. Diagrama de Contenedores (C2). SPA Vue 3, API NestJS, PostgreSQL 16, MinIO y Servidor de Correo.

5. Diagrama de Componentes (C3) — API REST Core

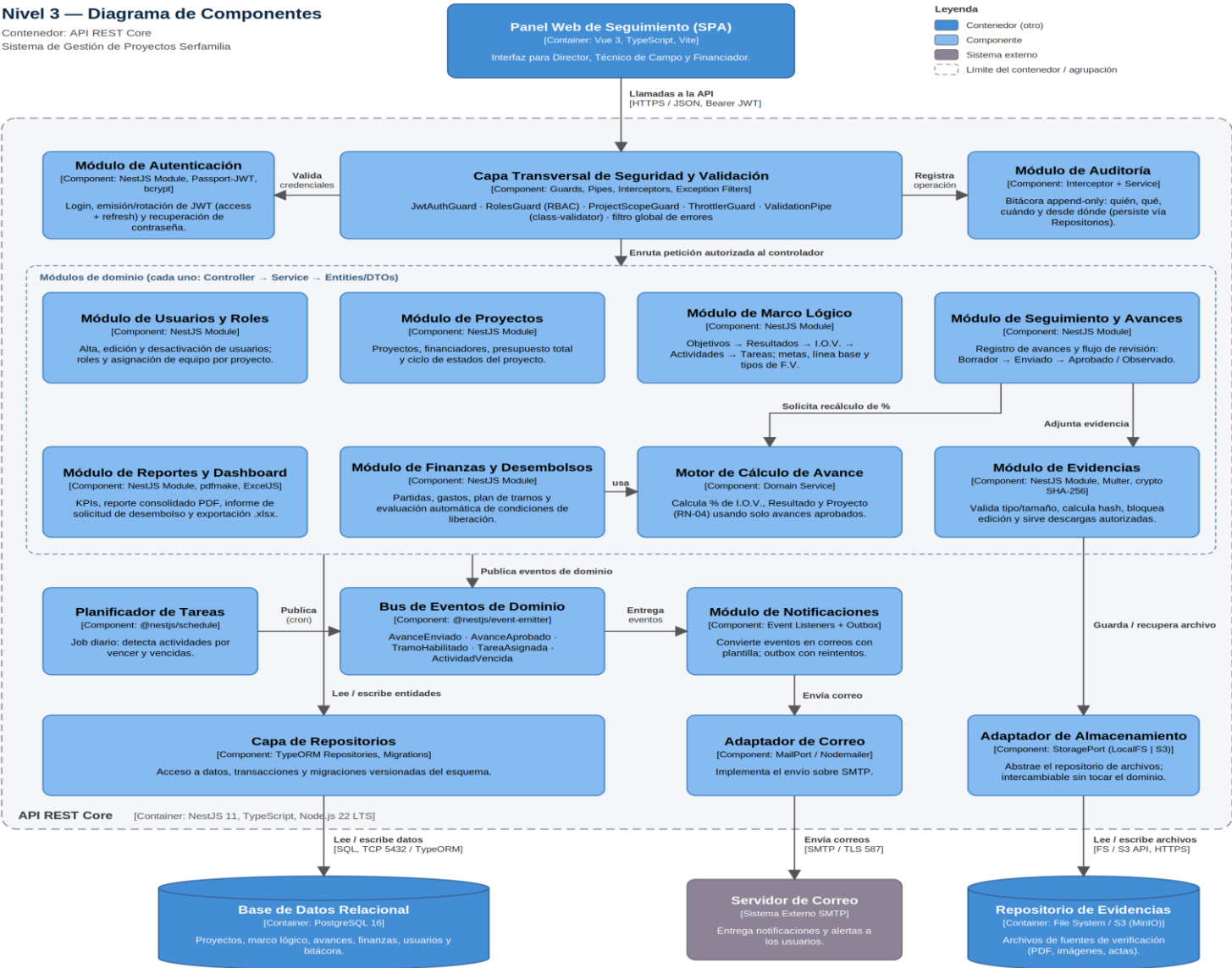


Figura 3. Diagrama C3 del contenedor API REST Core. Fuente editable: modelos/c4/c3-componentes.drawio

5.1 Descripción de los componentes

La API es un **monolito modular en capas**. Cada módulo de dominio de NestJS tiene un Controller (entrada HTTP), un Service (lógica de negocio) y sus repositorios TypeORM. Los componentes transversales — pipeline de seguridad, auditoría, evidencias y notificaciones — se comparten entre módulos, y los adaptadores (correo y almacenamiento) aíslan la aplicación de los servicios externos, lo que facilita cambiarlos sin tocar la lógica.

Componente	Tipo / Tecnología	Responsabilidad	Requisitos
Pipeline de peticiones	Guards, Pipes, Interceptors y Filters de NestJS	Punto de entrada común: JwtAuthGuard valida token, RolesGuard aplica RBAC, ThrottlerGuard limita tasa, ValidationPipe valida DTOs, AuditInterceptor captura escrituras, GlobalExceptionHandler normaliza errores (RFC 7807).	RF-07, RF-25, RF-19, RF-17, RF-18, RF-34, RNF-01, RNF-03, RNF-04
AuthController	NestJS Controller	Endpoints /auth: login, refresh, logout y recuperación de contraseña.	RF-17, RF-33
AuthService	NestJS Provider · Passport-JWT, bcrypt	Verifica credenciales, emite access token, rota y revoca refresh tokens, bloqueo por intentos fallidos.	RF-17, RF-32, RF-33, RNF-03, RNF-04
UsuariosController	NestJS Controller	Endpoints /usuarios para administración de cuentas y roles.	RF-32
UsuariosService	NestJS Provider	Reglas de cuentas (alta, baja lógica, cambio de rol) y consultas de asignación.	RF-23, RF-25, RF-18, RF-32
ProyectosController	NestJS Controller	Endpoints /proyectos: datos generales, presupuesto, equipo y cambio de estado.	RF-01, RF-02, RF-03, RF-04, RF-23
ProyectosService	NestJS Provider	Máquina de estados del proyecto (RN-09) y gestión de presupuesto y equipo.	RF-01, RF-02, RF-03, RF-23
MarcoLogicoController	NestJS Controller	Endpoints para resultados, I.O.V., actividades, tareas y cronograma.	RF-05, RF-06, RF-08, RF-09, RF-24, RF-13
MarcoLogicoService	NestJS Provider	Jerarquía (RN-01), validación de línea base/meta y fechas, estados de actividades.	RF-05, RF-06, RF-07, RF-08, RF-09, RF-10, RF-24, RF-25, RF-13, RF-15
ReportesController	NestJS Controller	Endpoints de panel KPI, reporte PDF y exportación XLSX.	RF-20, RF-21, RF-22
ReportesService	NestJS Provider · pdfmake, ExcelJS	Consolida métricas por fecha de corte y genera archivos (tareas pesadas vía cola).	RF-04, RF-20, RF-21, RF-22, RNF-02
FinanzasController	NestJS Controller	Endpoints de gastos, tramos y desembolsos.	RF-16, RF-29, RF-31
FinanzasService	NestJS Provider	Ejecución presupuestaria (RN-07), evaluación y habilitación de tramos (RN-06).	RF-16, RF-29, RF-30, RF-31
SeguimientoController	NestJS Controller	Endpoints de avances, envío, aprobación y observación.	RF-11, RF-26, RF-27, RF-28
SeguimientoService	NestJS Provider	Flujo de revisión (RN-03), cálculo de cumplimiento (RN-05), eventos de dominio.	RF-11, RF-12, RF-14, RF-26, RF-27, RF-28, RF-30
Capa de Repositorios	TypeORM Repository<T>	Acceso a datos por entidad, transacciones (QueryRunner) y migraciones versionadas.	RF-01, RF-02, RF-03, RF-23, RF-05, RF-06, RF-08, RF-09, RF-10, RF-24, RF-25, RF-11, RF-13, RF-14, RF-26, RF-27, RF-19, RF-16, RF-29, RF-30, RF-31, RF-17, RF-32, RF-34, RNF-01, RNF-07
AuditoriaService	NestJS Provider	Bitácora append-only y consulta filtrada.	RF-03, RF-27, RF-19, RF-34, RNF-05
AlertasScheduler	@nestjs/schedule	Cron diario: actividades vencidas e indicadores en riesgo.	RF-15, RF-35
NotificacionesService	@nestjs/event-emitter · pg-boss	Suscribe eventos de dominio, encola correos con reintentos y backoff exponencial.	RF-30, RF-33, RF-35

Componente	Tipo / Tecnología	Responsabilidad	Requisitos
EvidenciasService	Multer · crypto (SHA-256)	Valida tipo por magic bytes y tamaño, calcula hash, versiona, emite URLs prefirmadas.	RF-10, RF-12, RF-28, RF-19, RF-16, RF-31, RNF-02, RNF-05
MailAdapter	Nodemailer · Handlebars	Renderiza plantillas y envía por SMTP/TLS.	RF-33, RF-35
StorageAdapter	@aws-sdk/client-s3	Puerto de almacenamiento S3-compatible (MinIO / AWS S3).	RF-12, RNF-02, RNF-07

5.2 Interacciones clave en tiempo de ejecución

Escenario A — Registro de avance con evidencia (RF-11, RF-12, RF-26):

1. La SPA envía POST /avances (multipart) con el access token en la cabecera Authorization.
2. El Pipeline valida el token (JwtAuthGuard), verifica el rol Técnico (RolesGuard), aplica límite de tasa y valida el DTO.
3. SeguimientoController delega en SeguimientoService, que comprueba que la actividad o I.O.V. esté asignada al técnico (RN-10).
4. EvidenciasService valida tipo por magic bytes y tamaño (RN-04), calcula SHA-256 y, mediante StorageAdapter, sube el archivo a MinIO con nombre UUID.
5. En una sola transacción se guardan el avance (Borrador o Enviado) y los metadatos de la evidencia; AuditInterceptor registra la operación.
6. Si el avance se envía, SeguimientoService publica el evento «avance.enviado»; NotificacionesService encola el correo al Director en pg-boss y MailAdapter lo entrega vía SMTP.

Escenario B — Aprobación de avance y habilitación de tramo (RF-27, RF-14, RF-30):

1. El Director envía PATCH /avances/{id}/aprobar; el Pipeline verifica el rol Director.
2. SeguimientoService abre una transacción, cambia el estado a Aprobado (ADR-007) y recalcula el cumplimiento del I.O.V., del Resultado y del Proyecto (RN-05).
3. Dentro de la misma transacción consulta a FinanzasService, que evalúa las condiciones de los tramos pendientes en orden y marca como Habilitado el que las cumple (RN-06).
4. Al confirmar la transacción se publican los eventos «avance.aprobado» y «tramo.habilitado», que generan correos al técnico y al Director.

5.3 Estructura de código propuesta

La organización del repositorio refleja los componentes del diagrama, lo que mantiene la trazabilidad entre arquitectura y código:

```

api/src/
├── main.ts
├── app.module.ts
├── common/
├── config/                # validación de variables de entorno
├── database/migrations/   # migraciones TypeORM versionadas
├── modules/
│   ├── auth/ usuarios/ proyectos/ marco-logico/
│   ├── seguimiento/ finanzas/ reportes/
│   ├── evidencias/ notificaciones/ auditoria/ alertas/
│   └── (cada módulo: *.module.ts, *.controller.ts, *.service.ts, dto/, entities/)
├── infrastructure/
│   ├── storage/           # StorageAdapter (S3 / MinIO)
│   ├── mail/              # MailAdapter (Nodemailer)
│   └── queue/             # pg-boss

```

6. Registros de Decisiones Arquitectónicas (ADR)

Cada ADR sigue el formato de Michael Nygard (contexto, decisión y consecuencias) ampliado con la comparación de alternativas. Los ADR se versionan en docs/arquitectura/adr/ como archivos independientes; una decisión no se edita, sino que se reemplaza por un nuevo ADR con estado «Sustituye a ADR-xxx».

ADR	Decisión	Estado
ADR-001	Framework backend: NestJS	Aceptado
ADR-002	Estrategia de persistencia: PostgreSQL + TypeORM	Aceptado
ADR-003	Almacenamiento de evidencias en object storage S3-compatible	Aceptado
ADR-004	Autenticación JWT con refresh rotativo y autorización RBAC	Aceptado
ADR-005	Estilo arquitectónico: monolito modular	Aceptado
ADR-006	Procesamiento asíncrono y notificaciones con pg-boss	Aceptado
ADR-007	Máquinas de estado para avances, tramos y proyectos	Aceptado

ADR-001: Framework backend

Campo	Descripción
Estado	Aceptado
Fecha	Septiembre de 2026
Decisores	Arquitecto, Dev Backend, Gerente/SM
Requisitos relacionados	RNF-01, RNF-08, RF-18, RF-34

Contexto. La API concentra la lógica de negocio, la seguridad y las integraciones. El equipo tiene experiencia con TypeScript (que además es el lenguaje del frontend Vue 3), y el proyecto necesita autenticación, validación, módulos bien delimitados, tareas programadas y colas. Se evaluaron cuatro alternativas.

Alternativas evaluadas.

Alternativa	Ventajas	Desventajas
NestJS (Node.js)	Arquitectura modular con inyección de dependencias; guards, pipes e interceptors nativos; integración oficial con TypeORM, Swagger, schedule y event-emitter; mismo lenguaje que el frontend.	Rendimiento inferior en tareas intensivas de CPU; dependencia de decoradores y reflect-metadata.
Express.js puro	Mínimo y flexible; curva de aprendizaje baja.	Sin estructura impuesta: cada desarrollador organiza a su manera, lo que perjudica la mantenibilidad (RNF-08).
ASP.NET Core 8	Alto rendimiento; ecosistema maduro.	Implica un segundo lenguaje (C#); los tipos no se comparten con la SPA Vue.
Spring Boot 3	Robusto y estándar en entornos empresariales.	Mayor consumo de memoria; curva de aprendizaje que excede el tiempo disponible (~aprox. 2 meses).

Decisión. Se adopta **NestJS 11 sobre Node.js 22 LTS** con TypeScript en modo estricto. Los tipos de los DTO se exportan como contrato OpenAPI para generar los tipos del frontend.

Consecuencias positivas.

- Un solo lenguaje en todo el sistema reduce el costo de cambio de contexto y permite tipos compartidos.
- La estructura Module/Controller/Service se corresponde directamente con los componentes del C3.
- Guards e interceptors resuelven de forma declarativa RBAC y auditoría (RF-18, RF-34).

Consecuencias negativas y trade-offs.

- La generación de PDF es intensiva en CPU: se ejecuta como trabajo en cola (ADR-006) para no bloquear el event loop.

ADR-002: Estrategia de persistencia

Campo	Descripción
Estado	Aceptado
Fecha	Septiembre de 2026
Decisores	Arquitecto, Dev Backend, Gerente/SM
Requisitos relacionados	RF-01 a RF-16, RF-29 a RF-31, RNF-05, RNF-07, RNF-10

Contexto. El dominio es fuertemente jerárquico y relacional (Proyecto → Resultado → I.O.V./Actividad → Avance → Evidencia) y exige integridad referencial. Operaciones como aprobar un avance, recalcular el cumplimiento y habilitar un tramo deben ser atómicas. Se manejan montos monetarios y se requiere auditoría con valores anteriores y nuevos.

Alternativas evaluadas.

Alternativa	Ventajas	Desventajas
PostgreSQL + TypeORM	ACID, claves foráneas, NUMERIC exacto, JSONB para auditoría; integración nativa @nestjs/typeorm; ya definido en C2.	TypeORM tiene particularidades (relaciones lazy, generación de migraciones) que exigen disciplina.
PostgreSQL + Prisma	Excelente experiencia de desarrollo y tipado.	Integración menos directa con el patrón Repository de NestJS; cambio respecto a C2.
MongoDB + Mongoose	Esquema flexible y documentos anidados.	Integridad referencial y transacciones multi-documento más débiles; no encaja con un dominio relacional.
SQL Server	Robusto; ampliamente usado en entornos corporativos.	Costo de licencia en producción; menor afinidad con el ecosistema Node.

Decisión. Se adopta **PostgreSQL 16 con TypeORM** (patrón Data Mapper + Repository) con estas reglas: synchronize desactivado y cambios de esquema solo mediante migraciones versionadas en Git; claves primarias UUID; montos en NUMERIC(14,2); ON DELETE RESTRICT en relaciones del marco lógico; baja lógica (deleted_at) para usuarios y proyectos; tabla audit_log con columnas JSONB para los valores anterior y nuevo; los archivos de evidencia no se guardan en la BD (ADR-003).

Consecuencias positivas.

- Las reglas RN-01, RN-06 y RN-07 se aplican con transacciones y restricciones en la propia BD, no solo en código.
- Motor de código abierto, sin costo de licencia, con respaldos estándar (pg_dump / WAL).
- Las migraciones permiten reproducir el esquema en cualquier entorno (RNF-09).

Consecuencias negativas y trade-offs.

- Se requiere conocer las particularidades de TypeORM y revisar las migraciones generadas en cada Pull Request.
- Los cálculos de cumplimiento en consultas agregadas grandes se deben indexar adecuadamente (RNF-10).

ADR-003: Almacenamiento de evidencias

Campo	Descripción
Estado	Aceptado
Fecha	Septiembre de 2026
Decisores	Arquitecto, Dev Backend, Gerente/SM
Requisitos relacionados	RF-12, RF-19, RF-31, RNF-02, RNF-05, RNF-10

Contexto. Cada avance exige fuentes de verificación (PDF e imágenes de hasta 25 MB) que deben conservarse íntegras durante años y descargarse por el financiador. El C2 dejaba abierta la opción «File System local o Cloud Storage». Se estima un volumen de hasta 500 GB (RNF-10).

Alternativas evaluadas.

Alternativa	Ventajas	Desventajas
BYTEA en PostgreSQL	Una sola fuente de datos; transacción única.	La BD crece rápidamente; respaldos y restauraciones lentos; afecta el rendimiento.
Sistema de archivos local	Simple y sin componentes adicionales.	Difícil de escalar y replicar; acopla la API a un disco concreto; los permisos y respaldos se gestionan a mano.
Object storage S3-compatible (MinIO)	Diseñado para archivos grandes; URLs prefirmadas; versionado de objetos; migrable a AWS S3 o similar sin cambiar código.	Un contenedor más que operar; consistencia entre BD y objeto a cargo de la aplicación.

Decisión. Se adopta **MinIO (S3-compatible)** accedido exclusivamente a través del puerto **StorageAdapter**. En PostgreSQL se guardan solo los metadatos (nombre original, tipo MIME, tamaño, hash SHA-256, versión, autor). Los objetos se nombran con UUID, el bucket es privado y las descargas usan URLs prefirmadas de 5 minutos. El versionado de objetos del bucket se activa para reforzar RN-08.

Consecuencias positivas.

- La BD se mantiene liviana y los respaldos de archivos y datos son independientes.
- El hash SHA-256 permite demostrar integridad ante el financiador (RNF-05).
- Cambiar de MinIO a un proveedor en la nube solo requiere configuración.

Consecuencias negativas y trade-offs.

- Riesgo de inconsistencia si falla la transacción después de subir el archivo: se mitiga subiendo primero a un prefijo temporal, confirmando al guardar el avance y limpiando huérfanos con un trabajo diario.
- El equipo debe operar y respaldar un servicio adicional.

ADR-004: Autenticación y autorización

Campo	Descripción
Estado	Aceptado
Fecha	Septiembre de 2026
Decisores	Arquitecto, Dev Backend, Gerente/SM
Requisitos relacionados	RF-17, RF-18, RF-25, RF-32, RF-33, RNF-03, RNF-04

Contexto. Hay tres roles con permisos muy distintos y además reglas a nivel de recurso: un técnico solo opera sobre lo asignado y un financiador solo ve sus proyectos (RN-10). La API es stateless y la consume una SPA. Debe poder revocarse el acceso al desactivar un usuario (RF-32).

Alternativas evaluadas.

Alternativa	Ventajas	Desventajas
Sesiones en servidor (cookie de sesión)	Revocación inmediata; modelo conocido.	Requiere almacén de sesiones compartido; menos natural para una API REST stateless.
JWT de larga duración	Simple de implementar.	Imposible revocar antes de su expiración; alto impacto si se filtra.
JWT corto + refresh token rotativo	API stateless; revocación a nivel de refresh; riesgo acotado.	Mayor complejidad (rotación, detección de reutilización).
Proveedor de identidad (Keycloak/Auth0)	Funciones avanzadas (SSO, MFA).	Sobredimensionado para una ONG; costo u operación adicional.

Decisión. Se adopta **access token JWT de 15 minutos** (en memoria de la SPA) y **refresh token rotativo de 7 días** en cookie HttpOnly, Secure y SameSite=Strict, almacenado con hash en la BD para poder revocarlo; la reutilización de un refresh token ya rotado revoca toda la familia. Contraseñas con **bcrypt** (costo ≥ 12). La autorización combina **RBAC** declarativo (@Roles + RolesGuard) con políticas de recurso en los servicios (asignación del técnico, financiador del proyecto).

Consecuencias positivas.

- Cumple RNF-03 y RNF-04 con mecanismos estándar y auditables.
- Desactivar un usuario invalida su sesión en máximo 15 minutos (o de inmediato en el siguiente refresh).
- Los permisos quedan explícitos en el código, facilitando pruebas de seguridad (CP-18).

Consecuencias negativas y trade-offs.

- Mayor complejidad que una sesión simple; se requieren pruebas específicas de rotación.
- MFA queda fuera de la v1.0; puede agregarse después sin cambiar el modelo.

ADR-005: Estilo arquitectónico de la API

Campo	Descripción
Estado	Aceptado
Fecha	Septiembre de 2026
Decisores	Arquitecto, Dev Backend, Gerente/SM
Requisitos relacionados	RNF-07, RNF-08, RNF-09, RNF-10

Contexto. El sistema tiene siete áreas funcionales, un equipo de tres integrantes con un plazo de aproximadamente dos meses, un único cliente con carga moderada (≤ 100 usuarios) y presupuesto de infraestructura limitado. Se busca separar responsabilidades sin la complejidad operativa de un sistema distribuido.

Alternativas evaluadas.

Alternativa	Ventajas	Desventajas
Monolito en capas sin módulos	Muy simple al inicio.	Con el tiempo cualquier servicio accede a cualquier tabla; alto acoplamiento.
Monolito modular	Un solo despliegue; límites claros por dominio; transacciones locales; extraíble a servicios si hiciera falta.	Requiere disciplina para respetar los límites entre módulos.
Microservicios	Escalado y despliegue independientes.	Red, consistencia eventual, observabilidad y DevOps muy costosos para el tamaño del proyecto.

Decisión. Se adopta un **monolito modular**: un módulo NestJS por dominio (auth, usuarios, proyectos, marco-lógico, seguimiento, finanzas, reportes) más módulos transversales. Reglas: un módulo solo accede a sus propios repositorios; la colaboración entre módulos se hace mediante servicios exportados explícitamente o eventos de dominio; la regla se verifica en CI con una restricción de dependencias (RNF-08).

Consecuencias positivas.

- Un solo artefacto que desplegar y monitorear (RNF-09).
- La transacción de aprobación y habilitación de tramo es local y atómica (ADR-007).
- Los límites de módulo coinciden con los componentes del C3 y con las épicas de Jira.

Consecuencias negativas y trade-offs.

- Escala como una unidad; aceptable para la carga prevista (RNF-10).
- Un error grave puede afectar a todo el servicio; se mitiga con pruebas y health checks.

ADR-006: Procesamiento asíncrono y notificaciones

Campo	Descripción
Estado	Aceptado
Fecha	Septiembre de 2026
Decisores	Arquitecto, Dev Backend, Gerente/SM
Requisitos relacionados	RF-15, RF-20, RF-30, RF-33, RF-35, RNF-01, RNF-02

Contexto. El envío de correos y la generación de reportes PDF no deben bloquear la respuesta HTTP ni perderse si el servidor SMTP falla. Se necesitan reintentos y tareas programadas (alertas diarias). El C2 no incluye un contenedor de colas o caché.

Alternativas evaluadas.

Alternativa	Ventajas	Desventajas
Envío síncrono en la petición	Sin infraestructura adicional.	La latencia depende del SMTP; si falla, la notificación se pierde.
BullMQ + Redis	Colas de alto rendimiento y maduras.	Agrega un contenedor (Redis) que operar y respaldar.
pg-boss sobre PostgreSQL	Cola durable con reintentos en la BD existente; los trabajos pueden crearse en la misma transacción del negocio.	Menor rendimiento que Redis; suficiente para < 1 000 trabajos/día.

Decisión. Se adopta `@nestjs/event-emitter` para eventos de dominio en proceso y `pg-boss` como cola durable (3 reintentos con backoff exponencial) para correos y reportes pesados; `@nestjs/schedule` ejecuta el cron diario de alertas. `NotificacionesService` expone una interfaz propia para que la cola pueda sustituirse sin afectar a los módulos de dominio.

Consecuencias positivas.

- No se agrega infraestructura: se reutiliza PostgreSQL.
- Garantía de entrega: los trabajos persisten ante caídas del SMTP o reinicios (CP-35).
- Las respuestas HTTP no esperan al envío de correos (RNF-01).

Consecuencias negativas y trade-offs.

- Si la carga creciera de forma significativa, se reemplazaría `pg-boss` por `BullMQ + Redis` mediante un nuevo ADR; el cambio queda acotado a la infraestructura.
- La cola comparte recursos con la BD transaccional; se monitorea su tamaño.

ADR-007: Máquinas de estado para avances, tramos y proyectos

Campo	Descripción
Estado	Aceptado
Fecha	Septiembre de 2026
Decisores	Arquitecto, Dev Backend, Gerente/SM
Requisitos relacionados	RF-03, RF-14, RF-26, RF-27, RF-29, RF-30, RF-31, RNF-11

Contexto. El núcleo del negocio es que el financiador libera dinero según el avance verificado. Si los estados se modificaran libremente, podría aprobarse un avance sin evidencia o habilitarse un tramo sin cumplir sus condiciones. Se necesita una lógica explícita, auditable y fácil de probar.

Alternativas evaluadas.

Alternativa	Ventajas	Desventajas
Campo de estado libre	Implementación inmediata.	Transiciones inválidas posibles; reglas dispersas en el código.
Motor BPMN (Camunda u otro)	Modelado visual y flujos complejos.	Infraestructura y curva de aprendizaje desproporcionadas para flujos de 4–5 estados.
Máquina de estados en código con tabla de transiciones	Explícita, testeable con pruebas unitarias, sin dependencias externas.	Cambios de flujo requieren despliegue.

Decisión. Se adopta una **máquina de estados en código** por entidad, definida como tabla de transiciones permitidas por rol y validada en el servicio antes de persistir. Avance: Borrador → Enviado → Aprobado | Observado; Observado → Enviado; cualquier estado → Anulado (solo Director). Tramo: Pendiente → Habilitado → Desembolsado. Proyecto: según RN-09. La aprobación de un avance, el recálculo del cumplimiento (valor persistido, no calculado en cada consulta) y la habilitación de tramos se ejecutan en **una única transacción**; toda transición queda en la bitácora.

Consecuencias positivas.

- Imposibilita estados inválidos (CP-03, CP-26, CP-27, CP-31).
- El cálculo persistido hace que listados y panel sean rápidos (RNF-01).
- La bitácora de transiciones respalda cada desembolso ante auditorías del financiador (RNF-11).

Consecuencias negativas y trade-offs.

- El valor de cumplimiento persistido debe recalcularse ante cualquier cambio de pesos o metas; se implementa un recálculo explícito en esos casos.
- Nuevos estados requieren modificar código, pruebas y este ADR.